

Mini-project 2 in Advanced Machine Learning (02460)

by Gustav (s214374), Mathias (s214422), Magnus (s224188) & Jacob (s243569) on 2 April 2026

Part A: Pull-back geodesics

We train a VAE with a standard Gaussian prior and Gaussian likelihood $p(\mathbf{x}|\mathbf{z})$ on a subset of non-binarised MNIST (3 classes, 2048 observations) with a two-dimensional latent space. The encoder and decoder are convolutional networks using the course-provided architecture with Softmax activations and batch normalisation, trained for 50 epochs with Adam, $\text{lr}=10^{-3}$, batch size 32, while adding Gaussian noise ($\sigma=0.05$) to inputs. The decoder mean $f: \mathbb{R}^2 \rightarrow \mathbb{R}^{784}$ induces a *pull-back metric* $G(z) = J_f(z)^\top J_f(z)$. Rather than computing the Jacobian explicitly, we approximate the continuous curve energy $\int \dot{c}^\top G(c) \dot{c} dt$ with a discrete sum

$$E(c) = \sum_{i=0}^{N-1} \|f(c(t_i)) - f(c(t_{i+1}))\|^2, \quad (1)$$

which sums squared ℓ_2 distances in output space between consecutively decoded points.

We represent a curve as $N_t=20$ optimisable interior points initialised by linear interpolation between two fixed endpoints, and minimise (1) with Adam, $\text{lr}=0.01$, 1000 steps, step-decay $\times 0.5$ every 300 steps. Figure 1 (left) shows 25 geodesics: curves bend around low-density regions between digit clusters, reflecting that the pull-back metric inflates distances where the decoder output changes rapidly. Across retrainings the qualitative shape of geodesics is preserved—curves consistently avoid inter-cluster gaps—but precise paths shift because the decoder f is non-identifiable: different random initialisations yield different latent cluster positions and local curvature, even at similar ELBO. Within-class pairs (e.g. two 0s) produce the most stable geodesics, since the decoder models the intra-class manifold consistently; cross-class pairs show greater variation as the decision-boundary region changes shape between runs.

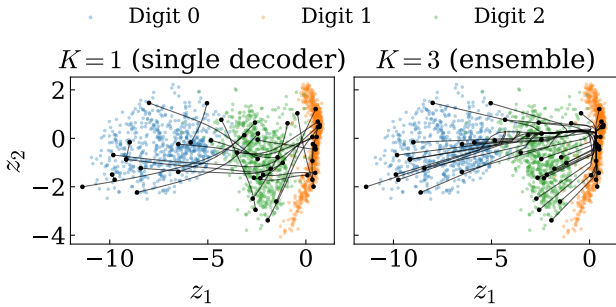


Figure 1: Geodesics between 25 random latent-variable pairs. Left: single decoder ($K=1$). Right: ensemble ($K=3$). Points coloured by digit class.

Part B: Ensemble VAE geometry

A single decoder’s geometry is unreliable: retraining changes f and thus G . We use an ensemble of K independently trained decoders. The *model-average curve energy* is

$$\mathcal{E}(c) \approx \sum_{i=0}^N \mathbb{E}_{l,k} [\|f_l(c(t_i)) - f_k(c(t_{i+1}))\|^2], \quad (2)$$

where f_l, f_k are drawn uniformly and independently from the K decoders (including $l=k$). When $l \neq k$, the cross-decoder term penalises decoder *disagreement*, inflating energy in uncertain regions and steering geodesics towards areas where the ensemble is confident. We approximate (2) with $S=50$ Monte Carlo (l, k) samples. Figure 1 (right) shows ensemble geodesics for the same 25 pairs.

We train $M=10$ VAEs and, for each, compute Euclidean and geodesic distances between fixed test-point pairs, encoding with the retrain’s own encoder. The CoV for pair (i, j) is $\text{CoV}_{ij} = \sigma\{d_{ij}^{(m)}\}_{m=1}^M / \mu\{d_{ij}^{(m)}\}_{m=1}^M$, averaged over 10 pairs. Figure 2 shows that Euclidean CoV (0.25) is constant w.r.t. K , as expected. Geodesic CoV at $K=1$ is lowest (0.20), increases sharply at $K=2$ (0.58), then drops to 0.31 at $K=3$. The low $K=1$ value reflects that a single decoder yields a deterministic metric per model, so geodesic distances track Euclidean distances closely. At $K=2$, the cross-decoder energy terms ($l \neq k$) introduce large variance: with only two decoders and thus a single cross-pair, the magnitude of their disagreement fluctuates strongly across retrainings. At $K=3$, nine (l, k) pairs provide better Monte Carlo coverage and the disagreement averages out, reducing CoV—though not yet below $K=1$. We expect that with larger K the trend would continue downward, and that $M=10$ retrainings may be insufficient to resolve the full asymptotic behaviour.

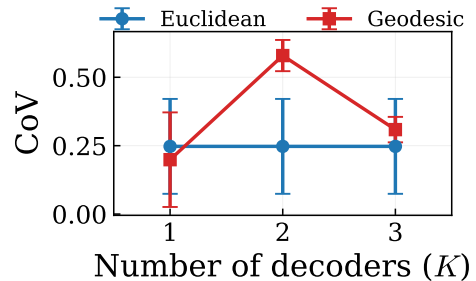


Figure 2: Mean CoV (± 1 std) of Euclidean and geodesic distances vs. number of ensemble decoders ($M=10$ retrainings, 10 pairs).

Code snippets

Curve energy (Eq. 1). Decodes all curve points and sums squared output-space differences between consecutive points.

```
1 def curve_energy(curve_points, decoder):
2     decoded = decoder(curve_points).mean                # (N+1, 1, 28, 28)
3     decoded_flat = decoded.reshape(len(decoded), -1)    # (N+1, 784)
4     diffs = decoded_flat[1:] - decoded_flat[:-1]
5     return (diffs ** 2).sum()
```

Ensemble curve energy (Eq. 2). Pre-decodes with all K decoders, then samples (l, k) pairs uniformly for Monte Carlo estimation.

```
1 def ensemble_curve_energy(curve_points, decoders, S=50):
2     K = len(decoders)
3     # Pre-decode all points with all decoders: (K, N+1, D)
4     all_dec = torch.stack([d(curve_points).mean.reshape(
5         len(curve_points), -1) for d in decoders])
6     l = torch.randint(0, K, (S,)) # MC sample decoder indices
7     k = torch.randint(0, K, (S,))
8     diffs = all_dec[l, :-1] - all_dec[k, 1:] # (S, N, D)
9     return (diffs ** 2).sum(-1).mean(0).sum() # avg over S, sum segments
```

Geodesic computation. Interior points initialised as linear interpolation, optimised with Adam to minimise curve energy.

```
1 def compute_geodesic(z0, z1, decoders, T=20, lr=0.01, steps=1000):
2     ts = torch.linspace(0, 1, T + 2)[1:-1]
3     interior = nn.Parameter(z0 + ts[:, None] * (z1 - z0)) # (T, 2)
4     opt = torch.optim.Adam([interior], lr=lr)
5     sched = StepLR(opt, step_size=300, gamma=0.5)
6     for _ in range(steps):
7         opt.zero_grad()
8         curve = torch.cat([z0[None], interior, z1[None]])
9         energy = ensemble_curve_energy(curve, decoders)
10        energy.backward()
11        opt.step(); sched.step()
12    return torch.cat([z0[None], interior.data, z1[None]])
```

Coefficient of Variation. Per-pair CoV across M retrainings, then averaged.

```
1 # dists: (M, num_pairs) -- geodesic or Euclidean distances
2 cov_per_pair = dists.std(axis=0) / (dists.mean(axis=0) + 1e-10)
3 mean_cov = cov_per_pair.mean() # reported in Figure 2
```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Claude (via GitHub Copilot)

Describe how the tools were used:

What did you use the tool(s) for? Claude was used via GitHub Copilot for code scaffolding (training loops, batch scripts) and L^AT_EX formatting assistance. All mathematical formulations and experimental design decisions were made by the authors.

At what stage(s) of the process did you use the tool(s)? During implementation and report writing.

How did you use or incorporate the generated output? Generated code was reviewed, tested, and modified to fit the project. Report text was drafted by the authors with formatting assistance.

References

All methods and notation follow the materials provided in weeks 5–8 of the Advanced Machine Learning course (02460) at DTU, spring 2026.